

Updated Route Logic (src/routes/your-thing.js)

This updated endpoint encapsulates the **Parse, Validate, Repair Once, and Quarantine** logic. It treats the model as an untrusted external entity and guarantees that your system never returns raw text to the client.

javascript

```
import express from 'express';
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
import { z } from 'zod';
import { JobCardOutputSchema, stubResponse } from '../llm/schema.js';

const router = express.Router();
const __dirname = path.dirname(fileURLToPath(import.meta.url));
const PROMPT_VERSION = "job-classifier-v1";

const RequestInputSchema = z.object({
  rawText: z.string({ required_error: "rawText is required" }).max(5000),
  userId: z.string({ required_error: "userId is required" })
});

// Helper: Ensure directories and log files exist cleanly
const quarantinePath = path.join(__dirname, '../..../logs/quarantine.jsonl');
if (!fs.existsSync(path.dirname(quarantinePath))) {
  fs.mkdirSync(path.dirname(quarantinePath), { recursive: true });
}

async function getSystemPrompt() {
  const promptPath = path.join(__dirname, `../../prompts/${PROMPT_VERSION}.md`);
  return fs.promises.readFile(promptPath, 'utf-8');
}

// Helper: Safely isolate, clean up code fences, and parse JSON structures
function cleanAndParseJSON(rawText) {
  let cleanText = rawText.trim();
  if (cleanText.includes("```")) {
    const match = cleanText.match(/```(?:json)?\s*([\s\S]*?)\s*```/);
    if (match) cleanText = match[1].trim();
  }
}
```

```
    return JSON.parse(cleanText);
  }

  // Helper: Append invalid items out of the system flow safely into quarantine
  async function quarantineFailure(inputData, rawModelOutput, errorDetails) {
    const logEntry = JSON.stringify({
      timestamp: new Date().toISOString(),
      promptVersion: PROMPT_VERSION,
      input: inputData,
      rawOutput: rawModelOutput,
      error: errorDetails
    });
    await fs.promises.appendFile(quarantinePath, logEntry + '\n', 'utf-8');
  }

  // Helper: Standardized OpenRouter API Invocation Execution Block
  async function callModel(systemPrompt, userPayload, messagesHistory = []) {
    const messages = [
      { role: "system", content: systemPrompt },
      { role: "user", content: JSON.stringify({ content: userPayload }) },
      ...messagesHistory
    ];

    const response = await fetch("https://openrouter.ai", {
      method: "POST",
      headers: {
        "Authorization": `Bearer ${process.env.OPENROUTER_API_KEY}`,
        "Content-Type": "application/json",
        "HTTP-Referer": "http://localhost:3000",
        "X-Title": "SaaS Robust Classifier"
      },
      body: JSON.stringify({
        model: "google/gemini-2.5-flash",
        temperature: 0.1, // Low variation for strict output matches
        messages: messages
      })
    });

    if (!response.ok) {
      throw new Error(`OpenRouter network failure: ${await response.text()}`);
    }

    const data = await response.json();
    return data.choices[0].message.content;
  }
```

```

router.post('/your-thing', async (expressReq, expressRes) => {
  let validatedInput;
  try {
    validatedInput = RequestInputSchema.parse(expressReq.body);
  } catch (err) {
    return expressRes.status(400).json({ error: "Validation Failed", invalidFields: err.er
  }

  if (process.env.LLM_STUB === '1') {
    return expressRes.status(200).json(stubResponse);
  }

  let systemPrompt;
  try {
    systemPrompt = await getSystemPrompt();
  } catch (err) {
    return expressRes.status(500).json({ error: "Prompt specification file missing" });
  }

  let rawModelOutput = "";
  let errorSummary = "";

  try {
    // Attempt 1: Initial Processing Execution
    rawModelOutput = await callModel(systemPrompt, validatedInput.rawText);

    try {
      const parsedJson = cleanAndParseJSON(rawModelOutput);
      const schemaCheck = JobCardOutputSchema.safeParse(parsedJson);

      if (schemaCheck.success) {
        return expressRes.status(200).json(schemaCheck.data); // Early exit on perfect str
      }
      errorSummary = JSON.stringify(schemaCheck.error.errors);
    } catch (parseErr) {
      errorSummary = `JSON Syntax Break: ${parseErr.message}`;
    }

    // Attempt 2: Bounded Repair Loop (Exactly Once)
    const repairHistory = [
      { role: "assistant", content: rawModelOutput },
      {
        role: "user",

```

```

    content: `Your previous answer was rejected for this reason: ${errorSummary}. Retu
  }
];

rawModelOutput = await callModel(systemPrompt, validatedInput.rawText, repairHistory);

const repairParsedJson = cleanAndParseJSON(rawModelOutput);
const repairSchemaCheck = JobCardOutputSchema.safeParse(repairParsedJson);

if (repairSchemaCheck.success) {
  return expressRes.status(200).json(repairSchemaCheck.data);
}

errorSummary = JSON.stringify(repairSchemaCheck.error.errors);

} catch (globalExecutionErr) {
  errorSummary = `Fatal runtime processing exception: ${globalExecutionErr.message}`;
}

// Final Demarcation: Complete failure across execution boundaries -> Quarantine and 422
await quarantineFailure(validatedInput, rawModelOutput, errorSummary);
return expressRes.status(422).json({
  error: "Unprocessable Entity",
  message: "Model execution failed to adhere to the schema contract after structural rep
});
});

export default router;

```

Use code with caution.

2. Testing Execution Commands and Expected Output

Execute these terminal calls from your terminal to verify validation, structural success bounds, and isolated quarantine logging.

Test 1: Valid Structural Output Processing (200 OK Execution)

Command:

bash

```
curl -X POST http://localhost:3000/your-thing \  
  -H "Content-Type: application/json" \  
  -d '{"rawText": "The analytical engine is throwing a database deadlock error when refres
```

Use code with caution.

Expected JSON Response Body (200 OK):

json

```
{  
  "status": "success",  
  "confidence": 0.98,  
  "extractedData": {  
    "title": "Database deadlock error on dashboard refresh",  
    "tags": ["analytical-engine", "database", "deadlock", "dashboard"],  
    "priority": "high"  
  },  
  "summary": "User is experiencing persistent database deadlocks explicitly when completin  
}
```

Use code with caution.

Test 2: Invalid Broken Input Field Structure (400 Bad Request Execution)

Command:

bash

```
curl -X POST http://localhost:3000/your-thing \  
  -H "Content-Type: application/json" \  
  -d '{"userId": "user_101"}'
```

Use code with caution.

Expected JSON Response Body (400 Bad Request):

json

```
{
  "error": "Validation Failed",
  "invalidFields": [
    {
      "code": "invalid_type",
      "expected": "string",
      "received": "undefined",
      "path": ["rawText"],
      "message": "rawText is required"
    }
  ]
}
```

Use code with caution.

Test 3: Unresolvable Schema Invalidation (422 Quarantine Verification Execution)

To intentionally break validation, temporarily edit the `priority` enum values in your prompt at `prompts/job-classifier-v1.md` to require an unmapped attribute string like `"critical-omega"`.

Command:

bash

```
curl -X POST http://localhost:3000/your-thing \
  -H "Content-Type: application/json" \
  -d '{"rawText": "Force an output generation with high priority values.", "userId": "user
```

Use code with caution.

Expected JSON Response Body (422 Unprocessable Entity):

json

```
{  
  "error": "Unprocessable Entity",  
  "message": "Model execution failed to adhere to the schema contract after structural rep  
}
```

Use code with caution.

Inspecting the Resulting Output Log inside `logs/quarantine.jsonl`:

bash

```
cat logs/quarantine.jsonl
```

Use code with caution.

Expected Stream File Entry Layout:

json

```
{"timestamp": "2026-08-25T11:53:22.104Z", "promptVersion": "job-classifier-v1", "input": {"rawT
```

Use code with caution.

(Once confirmed, reverse your prompt edits back to `'Low' | 'medium' | 'high'` to restore normal service configurations.)

I have fully answered this request. If you would like to proceed further, let me know if you want to explore the following steps:

- ↪ Write automated **Jest test configurations** to assert the repair retry loop works using mock server endpoints
- ↪ Configure an **untrusted text truncation workflow** to strip non-printable hex characters prior to parsing.